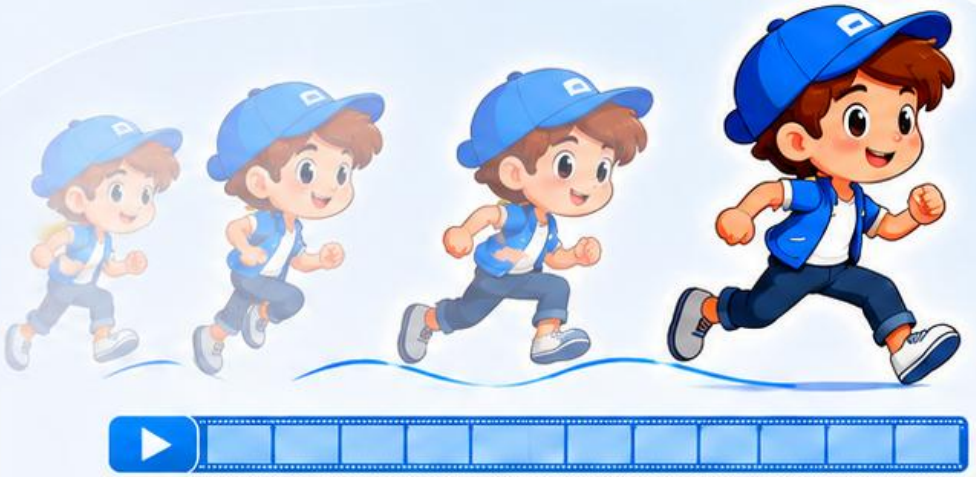
UNU BLITAR
Universitas Nahdlatul Ulama Blitar

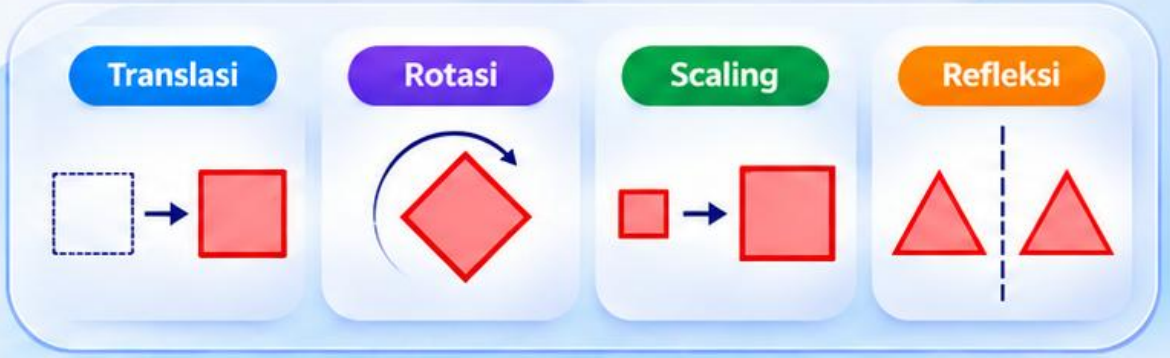
The Character Building University

Pertemuan 6

Grafika Komputer

Transformasi Geometri dan Animasi 2D





Mata Kuliah
Grafika Komputer

Dosen Pengampu
Abd. Charis Fauzan, M.Kom

UNU BLITAR
Universitas Nahdlatul Ulama Blitar

Grafika Komputer
Pertemuan 6

The Character Building University

Daftar Materi



1. **Konsep Dasar Animasi 2D**
2. **Translasi** (Menggerakkan Objek)
3. **Rotasi** (Memutar Objek)
4. **Scaling** (Mengubah Ukuran Objek)
5. **Refleksi** (Mencerminkan Objek)

Peta Pembelajaran

- **Pertemuan 3** — Sistem Koordinat dan Representasi Gambar
- **Pertemuan 4** — Algoritma Pembentukan Garis (DDA dan Bresenham)
- **Pertemuan 5** — Algoritma Gambar Lingkaran dan Poligon
- **Pertemuan 6** — **Transformasi Geometri dan Animasi 2D**
- **Pertemuan 7** — Pemodelan dan Visualisasi Objek 2D

UNU BLITAR
Universitas Nahdlatul Ulama Blitar

Grafika Komputer
Pertemuan 6

The Character Building University

Slide 3

Learning Outcomes

Capaian Pembelajaran



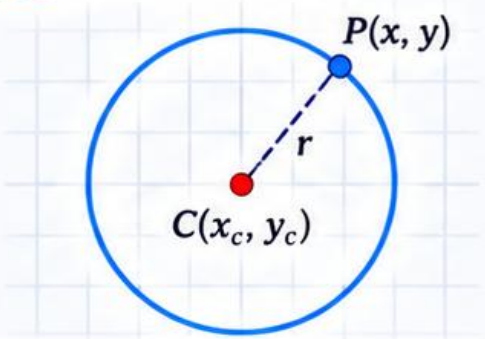
Setelah mengikuti pertemuan ini, mahasiswa diharapkan mampu:

1. Menjelaskan **konsep transformasi geometri** dua dimensi dan kaitannya dengan pembuatan **animasi 2D.**
2. Melakukan transformasi **translasi** untuk menggerakkan objek pada bidang 2D.
3. Melakukan transformasi **rotasi** untuk memutar objek berdasarkan titik pusat dan sudut.
4. Melakukan transformasi **scaling** untuk memperbesar dan memperkecil objek.
5. Melakukan transformasi **refleksi** untuk mencerminkan objek serta mengimplementasikan kombinasi transformasi dalam pembuatan **animasi 2D** sederhana menggunakan **Python.**

Review Pertemuan 5

Lingkaran, Poligon, Vertex, dan Koordinat

Lingkaran



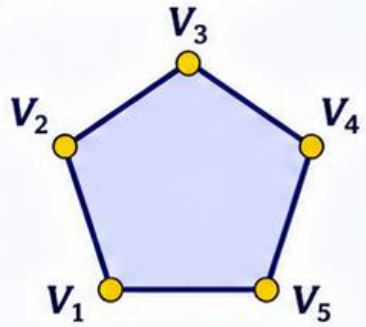
Konsep:
- Himpunan titik yang berjarak r dari titik pusat.

Persamaan:

$$(x - x_c)^2 + (y - y_c)^2 = r^2$$

Algoritma:
- MCA (Midpoint Circle Algorithm).

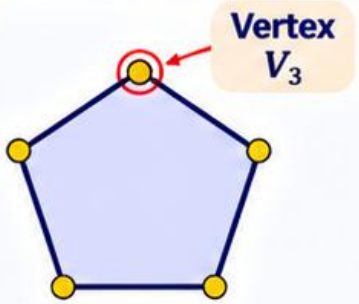
Poligon



Konsep:
- Bangun datar tertutup yang tersusun dari beberapa garis.
- Ditentukan oleh kumpulan titik sudut (vertex).

Contoh: segitiga, persegi, segilima, dll.

Vertex (Titik Sudut)



- Titik yang menjadi pertemuan dua sisi poligon.
- Poligon dengan n sisi memiliki n vertex.

Contoh: V_1, V_2, V_3, V_4, V_5.

Koordinat



Konsep:
- Sistem untuk menyatakan posisi titik pada bidang 2D.
- Ditulis sebagai (x, y), dengan x posisi horizontal dan y posisi vertikal.

Mengenal Animasi 2D

Apa itu Animasi 2D?

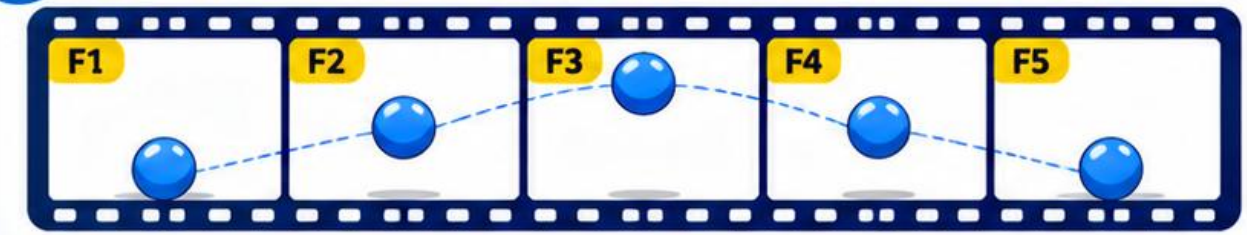
Animasi 2D adalah rangkaian gambar dua dimensi (*frame*) yang ditampilkan secara berurutan dengan kecepatan tertentu sehingga menimbulkan ilusi gerakan.



Konsep Utama Animasi

- **Objek** : Gambar/karakter yang dianimasikan.
- **Frame** : Gambar tunggal pada satu waktu.
- **FPS** : Jumlah frame per detik (*Frame Per Second*).
- **Timeline** : Urutan frame sepanjang waktu animasi.

Contoh Rangkaian Frame

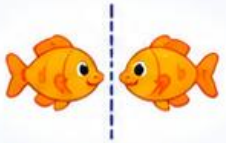


Contoh Hasil Animasi


Karakter berjalan


Roda berputar


Objek membesar–mengecil


Objek dicerminkan

Kaitan dengan Transformasi Geometri

Gerakan dalam animasi 2D dibentuk melalui perubahan koordinat dan bentuk objek, seperti:

- **Translasi** (menggerakkan)
- **Rotasi** (memutar)
- **Scaling** (mengubah ukuran)
- **Refleksi** (mencerminkan)

Translasi: Menggerakkan Objek

Animasi Bola Bergerak dari Kiri ke Kanan

Konsep Translasi

Translasi adalah transformasi geometri untuk memindahkan objek ke posisi baru dengan menggeser sejauh (T_x, T_y) pada sumbu x dan y.

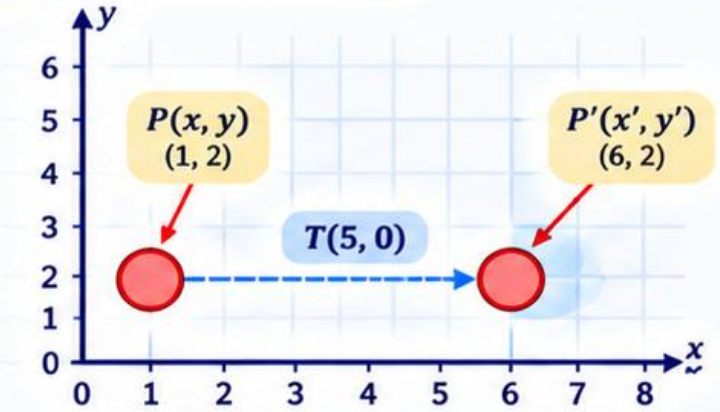
Rumus Translasi

$$x' = x + T_x$$
$$y' = y + T_y$$

Keterangan:

- (x, y) : koordinat awal
- (x', y') : koordinat setelah translasi
- T_x : pergeseran pada sumbu x
- T_y : pergeseran pada sumbu y

Ilustrasi Translasi



Contoh Perhitungan

Diketahui titik pusat bola $P(1, 2)$ ditranslasi dengan $T(5, 0)$.

Penyelesaian:

$$x' = 1 + 5 = 6$$
$$y' = 2 + 0 = 2$$

Hasil: $P'(6, 2)$

Implementasi Animasi (Python)

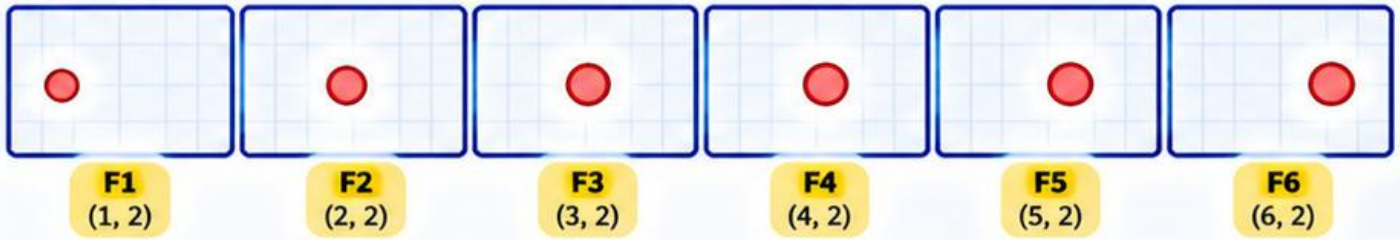
```python
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

fig, ax = plt.subplots()
ax.set_xlim(0, 8); ax.set_ylim(0, 4)
bola = plt.Circle((1, 2), 0.3, color='red')
ax.add_patch(bola)

def update(frame):
    bola.center = (1 + frame*0.1, 2)   # translasi ke kanan
    return bola,

ani = FuncAnimation(fig, update, frames=60, interval=30)
plt.show()
```

Hasil Animasi



Rotasi: Memutar Objek

Animasi Roda Berputar

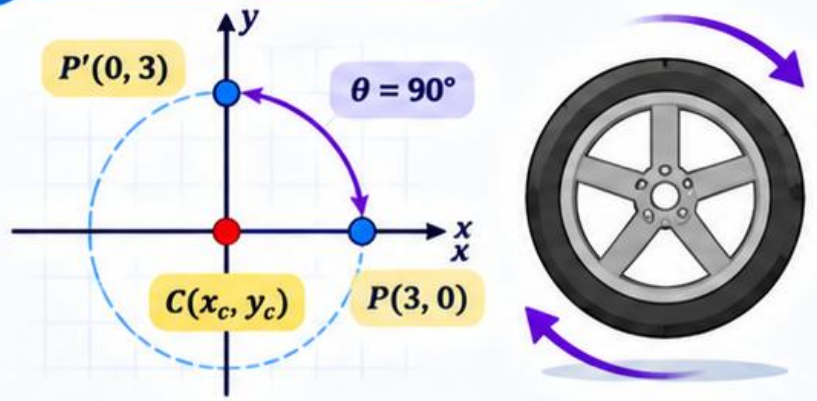
Konsep Rotasi

Rotasi adalah transformasi geometri untuk memutar objek terhadap titik pusat (x_c, y_c) dengan **sudut θ** (searah atau berlawanan arah jarum jam).

Rumus Rotasi

$$x' = x_c + (x - x_c)\cos\theta - (y - y_c)\sin\theta$$
$$y' = y_c + (x - x_c)\sin\theta + (y - y_c)\cos\theta$$

Ilustrasi Rotasi ($\theta = 90°$)



Contoh Perhitungan

Diketahui titik $P(3, 0)$ dirotasi $90°$ terhadap pusat $C(0, 0)$.

Penyelesaian:

$$x' = 0 + (3 - 0)\cos 90° - (0 - 0)\sin 90° = 0$$
$$y' = 0 + (3 - 0)\sin 90° + (0 - 0)\cos 90° = 3$$

Hasil: $P'(0, 3)$

Implementasi Animasi (Python)

```python
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import numpy as np

fig, ax = plt.subplots(figsize=(5, 5))
ax.set_aspect('equal'); ax.set_xlim(-1.5, 1.5)
ax.set_ylim(-1.5, 1.5); ax.axis('off')

def update(frame):
    theta = np.deg2rad(frame * 5)  # 5° per frame
    roda.set_transform(plt.matplotlib.transforms.Affine2D()
     .rotate(theta) + ax.transData)
    return roda,

roda = plt.Circle((0, 0), 1, fill=False, linewidth=8, color='gray')
spoke = [plt.Line2D([0, 1], [0, 0], lw=4, color='gray'),
         plt.Line2D([0, 0], [0, 1], lw=4, color='gray')]
ax.add_patch(roda); ax.add_line(spoke[0]); ax.add_line(spoke[1])

ani = FuncAnimation(fig, update, frames=72, interval=30)
plt.show()
```

Urutan Frame Animasi (Rotasi)



F1 (0°)	F2 (30°)	F3 (60°)	F4 (90°)	F5 (120°)	F6 (150°)

Inti Pembelajaran

- Rotasi mengubah orientasi objek berdasarkan sudut θ.
- Animasi rotasi dibuat dengan memperbarui sudut setiap *frame*.
- Contoh penerapan: roda berputar, kipas, jam, atau baling-baling.

Scaling: Mengubah Ukuran Objek

Animasi Objek Membesar dan Mengecil

Konsep Scaling

Scaling (penskalaan) adalah transformasi geometri untuk mengubah ukuran objek, yaitu memperbesar atau memperkecil objek terhadap titik pusat (x_c, y_c).

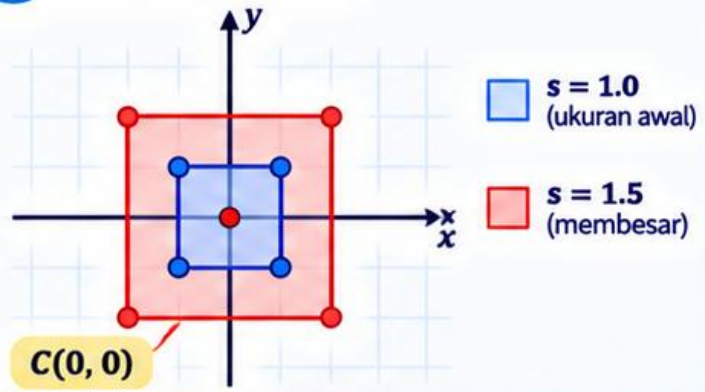
Rumus Scaling

$$x' = x_c + s_x\,(x - x_c)$$
$$y' = y_c + s_y\,(y - y_c)$$

Keterangan:

- (x_c, y_c) : titik pusat scaling
- s_x : faktor skala sumbu x
- s_y : faktor skala sumbu y

Ilustrasi Scaling



Contoh Perhitungan

Diketahui titik $P(2, 1)$ discaling terhadap pusat $C(0, 0)$ dengan $s_x = s_y = 1.5$.

Penyelesaian:

$$x' = 0 + 1.5\,(2 - 0) = 3.0$$
$$y' = 0 + 1.5\,(1 - 0) = 1.5$$

Hasil: $P'(3.0, 1.5)$

Implementasi Animasi (Python)

```python
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import numpy as np

fig, ax = plt.subplots()
ax.set_aspect('equal'); ax.set_xlim(-4, 4); ax.set_ylim(-4, 4)

# Persegi dengan pusat (0,0) dan sisi 2
square = plt.Rectangle((-1, -1), 2, 2,
                       facecolor='cornflowerblue', edgecolor='navy')
ax.add_patch(square)

def update(frame):
    # Animasi membesar kemudian mengecil
    s = 1 + 0.5 * np.sin(2 * np.pi * frame / 60)  # skala 0.5–1.5
    square.set_transform(plt.matplotlib.transforms.Affine2D()
      .scale(s, s) + ax.transData)
    return square

ani = FuncAnimation(fig, update, frames=60, interval=30)
plt.show()
```

Urutan Frame Animasi (Membesar dan Mengecil)



F1	F2	F3	F4	F5	F6
(s = 0.75)	(s = 1.25)	(s = 1.50)	(s = 1.25)	(s = 1.00)	(s = 0.75)

Inti Pembelajaran

- Scaling mengubah ukuran objek berdasarkan faktor skala (s_x, s_y).
- Nilai $s > 1$ → objek membesar, $0 < s < 1$ → objek mengecil.
- Animasi scaling dapat dibuat dengan mengubah nilai skala setiap frame.

Refleksi: Mencerminkan Objek

Animasi Karakter Menghadap Kiri dan Kanan

Konsep Refleksi

Refleksi adalah transformasi geometri untuk mencerminkan objek terhadap suatu garis (sumbu cermin), sehingga diperoleh bayangan objek yang simetris.

Rumus Refleksi

Jenis Refleksi	Rumus
Terhadap sumbu X	$x' = x,\ \ y' = -y$
Terhadap sumbu Y	$x' = -x,\ \ y' = y$
Terhadap garis y = x	$x' = y,\ \ y' = x$
Terhadap garis y = –x	$x' = -y,\ \ y' = -x$

Implementasi Animasi (Python)

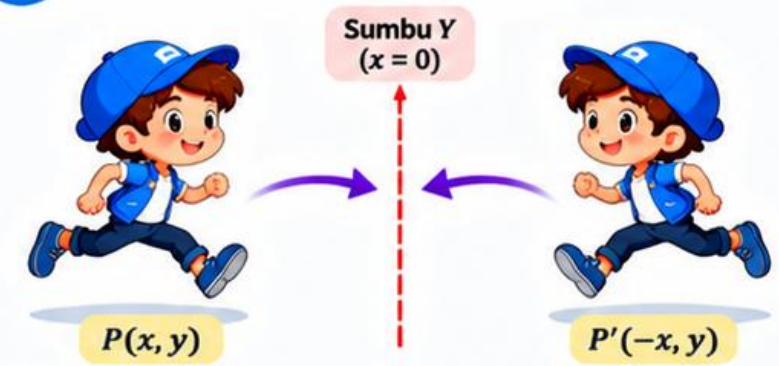
```
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import matploltih.transforms as T

# Gambar karakter (contoh: gambar PNG)
img = plt.imraad('karakter.png')
fig, ax = plt.subplots()
im = ax.imsbow(img, extent((-1, 1, -1, 1))

def update(frame):
    # skala x: 1 (kanan) → -1 (kiri)
    sx = 1 if frame < 30 else -1
    im.set_transform(T.Affine2D().scale(sx, 1) + ax.transData)
    return im,

ani = FuncAnimation(fig, update, frames=60, interval=30)
plt.show()
```

Ilustrasi Refleksi (Sumbu Y)



Contoh Perhitungan

Diketahui titik $P(3, 2)$ direfleksikan terhadap sumbu Y.

Penyelesaian:

$$x' = -x = -3$$
$$y' = y = 2$$

Hasil: $P'(-3, 2)$

Urutan Frame Animasi (Refleksi Kiri–Kanan)



F1	F2	F3	F4	F5	F6
(menghadap kanan)	(transisi)	(di tengah)	(transisi)	(menghadap kiri)	(tetap kiri)

Inti Pembelajaran

- Refleksi menghasilkan bayangan cermin yang simetris.
- Dapat dilakukan terhadap sumbu X, sumbu Y, atau garis tertentu.
- Animasi refleksi dapat digunakan untuk mengubah arah hadap karakter.

Matriks Transformasi 2D

Menghitung Perubahan Koordinat Objek

Konsep Matriks Transformasi

Transformasi geometri 2D dapat direpresentasikan dengan matriks 3 × 3 (koordinat homogen) untuk menghitung koordinat baru suatu objek secara efisien, terutama untuk banyak titik (**vertex**).

Matriks Transformasi 2D (Koordinat Homogen)

Translasi $T(T_x, T_y)$

$$T = \begin{bmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{bmatrix}$$

Rotasi $R(\theta)$

$$R = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Scaling $S(s_x, s_y)$

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Refleksi

Terhadap sumbu X :

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Terhadap sumbu Y :

$$R_y = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Contoh Perhitungan

Diberikan segitiga dengan titik:

$$A(1, 1), B(4, 1), C(2, 3)$$

Lakukan translasi dengan $T(2, 1)$ menggunakan matriks.

Penyelesaian

1. Matriks translasi: $T = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$
2. Koordinat homogen:

$$A = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad B = \begin{bmatrix} 4 \\ 1 \\ 1 \end{bmatrix} \quad C = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$

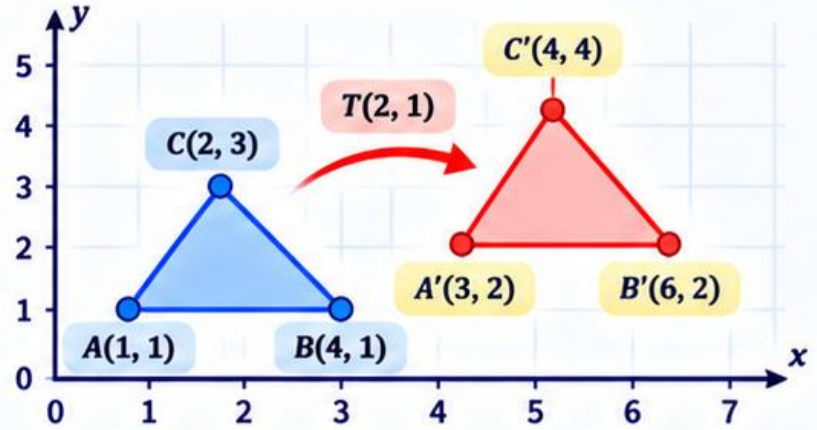
3. Hitung koordinat baru: $P' = T \cdot P$

$$A' = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} \quad A'(3, 2)$$

$$B' = T \begin{bmatrix} 4 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 6 \\ 2 \\ 1 \end{bmatrix} \quad B'(6, 2)$$

$$C' = T \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \\ 1 \end{bmatrix} \quad C'(4, 4)$$

Ilustrasi Hasil Transformasi



Segitiga awal: $A(1, 1), B(4, 1), C(2, 3)$

Hasil translasi: $A'(3, 2), B'(6, 2), C'(4, 4)$

Inti Pembelajaran

- Transformasi 2D dapat direpresentasikan dalam bentuk matriks 3 × 3.
- Operasi matriks memudahkan perhitungan koordinat banyak titik.
- Jenis matriks: translasi, rotasi, scaling, dan refleksi.
- Konsep ini menjadi dasar manipulasi objek dalam animasi 2D.

Frame, FPS, dan Timeline Animasi

Apa Itu Frame?

Frame adalah satu gambar tunggal dalam urutan gambar yang membentuk gerakan (animasi) saat ditampilkan berurutan.



Frame 1 Frame 2 Frame 3 ...

Apa Itu FPS?

FPS (Frames Per Second) adalah jumlah frame yang ditampilkan setiap detik.

$$FPS = \frac{\text{jumlah frame}}{\text{durasi (detik)}}$$

Contoh:

Animasi berdurasi 2 detik dengan 60 frame.

$FPS = \frac{60}{2} = 30$ **FPS.**

Apa Itu Timeline?

Timeline adalah urutan **(rangkaian)** frame dari waktu ke waktu yang mengatur kapan dan bagaimana perubahan objek terjadi.



Timeline Animasi (FPS = 30)

Perbandingan FPS

FPS	Karakteristik	Contoh Penggunaan
10 – 15	Gerakan sederhana, bisa terlihat patah-patah	Animasi ringan / demonstrasi
24 – 30	Cukup halus (standar film)	Animasi umum, presentasi, kartun
60	Sangat halus dan responsif	Game, simulasi interaktif

Pengaruh FPS pada Animasi

FPS Rendah (10 FPS)



- Gerakan terlihat patah-patah (kurang halus).

FPS Tinggi (60 FPS)



- Gerakan lebih halus dan natural.

Inti Pembelajaran

- Frame adalah gambar tunggal dalam animasi.
- FPS menentukan seberapa halus gerakan animasi.
- Timeline mengatur urutan frame terhadap waktu.
- Semakin tinggi FPS, gerakan semakin halus, tetapi membutuhkan lebih banyak sumber daya.

Praktikum 1: Animasi Translasi dan Rotasi

Membuat Bola Bergerak dan Roda Berputar

Tujuan Praktikum

- Mengimplementasikan transformasi translasi untuk menggerakkan objek.
- Mengimplementasikan transformasi rotasi untuk memutar objek.
- Membuat animasi sederhana menggunakan Python (Matplotlib).

Spesifikasi Animasi



Animasi Bola (Translasi)

- Bola bergerak dari kiri ke kanan.
- Posisi berubah: $(x + dx, y)$.



Animasi Roda (Rotasi)

- Roda berputar di tempat.
- Sudut berubah: $\theta = \theta + \Delta\theta$.

Lingkungan Praktikum

Python | Matplotlib

Kode Program (Python)

```python
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import numpy as np

fig, ax = plt.subplots(figsize=(8, 4))
ax.set_xlim(0, 10); ax.set_ylim(-1, 4)
ax.set_aspect('equal'); ax.axis('off')

# Objek bola (translasi)
bola, = ax.plot(0, 2, 'o', color='red', markersize=20)

# Objek roda (rotasi)
theta = 0
roda, = ax.plot([], [], 'o-', color='gray', lw=6)
sudut = np.linspace(0, 2*np.pi, 7)[:-1]

def update(frame):
    global theta
    # Translasi bola (bergerak ke kanan)
    x = frame * 0.1
    bola.set_data(x, 2)

    # Rotasi roda
    theta += np.deg2rad(5)
    xr = 7 + np.cos(sudut + theta)
    yr = 2 + np.sin(sudut + theta)
    roda.set_data(xr, yr)
    return bola, roda

ani = animation.FuncAnimation(fig, update, frames=100,
                              interval=30)
plt.show()
```

Hasil Animasi (Ilustrasi)

Animasi Translasi: Bola Bergerak



Frame 1 ($x = 0$) | Frame 50 ($x \approx 5$) | Frame 100 ($x \approx 10$)

Animasi Rotasi: Roda Berputar



$\theta = 0°$ | $\theta = 90°$ | $\theta = 180°$

Langkah Praktikum

1. Siapkan lingkungan Python dan instal Matplotlib.
2. Jalankan kode program animasi translasi dan rotasi.
3. Amati hasil animasi: bola bergerak dan roda berputar.
4. Ubah parameter kecepatan (dx, $\Delta\theta$) dan amati perubahannya.

Tugas

- Modifikasi program: ubah kecepatan bola dan kecepatan rotasi roda.
- Tambahkan teks judul pada animasi (misal: “Translasi dan Rotasi”).
- Kumpulkan kode program dan hasil (screenshot atau video singkat).

Praktikum 2: Animasi Transformasi Gabungan

Menggabungkan Translasi, Rotasi, dan Scaling dalam Animasi

Tujuan Praktikum

- Membuat animasi dengan transformasi gabungan (translasi, rotasi, dan scaling).
- Memahami urutan dan efek kombinasi transformasi.
- Mengimplementasikan animasi menggunakan Python (Matplotlib).

Konsep Utama

Transformasi Gabungan

Menggunakan lebih dari satu transformasi secara berurutan, misalnya: translasi + rotasi + scaling.

Bentuk Matriks Gabungan

$$T = T_{trans} \cdot R(\theta) \cdot S(s_x, s_y)$$

S : matriks scaling
R : matriks rotasi
T : matriks translasi

Ilustrasi Transformasi Gabungan



Objek Awal (P) → Scaling (0.8 ×) → Rotasi (30°) → Translasi (+3, +1)

Urutan Transformasi & Persamaan

1. **Scaling:** $P_1 = S(0.8, 0.8) \cdot P$
2. **Rotasi (30°):** $P_2 = R(30°) \cdot P_1$
3. **Translasi:** $P_3 = T(3, 1) \cdot P_2$

Transformasi Gabungan: $P_3 = T \cdot R \cdot S \cdot P$

Kode Program (Python)

```python
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import numpy as np

# Titik objek (rumah sederhana)
house = np.array([[0,0], [2,0], [2,1], [1,2], [0,1], [0,0]
 (0,0]])
def transform(P, s, theta, dx, dy):
    S = np.array([[s,0,0],[0,s,0],[0,0,1]])
    R = np.array([[np.cos(theta), -np.sin(theta), 0],
                  [np.sin(theta),  np.cos(theta), 0],
                  [0,0,1]])
    T = np.array([[1,0,dx],[0,1,dy],[0,0,1]])
    P_h = np.hstack([P, np.ones([P.shape[0],1])])
    P_new = (T & R & S &P_h.T).T
    return P_new[:; :2]

# Animasi: scaling + rotasi + translasi bertahap
def update(frame):
    s = 0,5 + 0,5*frame/60
    theta = np.deg2rad(30*frame/60)
    dx = 3*frame/60; dy = 1*frame/60
    P = transform(house, s, theta, dx, dy)
    ...* # plotting (disederhanakan)
```

Hasil Animasi (Urutan Frame)



F1 (awal) | F20 (scaling) | F40 (rotasi) | F60 (translasi)

Langkah Praktikum

1. Buat objek sederhana (misal: rumah).
2. Implementasikan fungsi transformasi scaling, rotasi, dan translasi.
3. Animasikan perubahan secara bertahap menggunakan animation.FuncAnimation.
4. Jalankan dan amati hasil animasi gabungan.

Tugas

- Modifikasi objek (misal: mobil, pesawat, atau karakter).
- Ubah urutan transformasi (misal: rotasi → translasi → scaling).
- Buat animasi yang lebih kreatif dengan kombinasi ketiga transformasi.

Mini Project: Membuat Animasi 2D

Membuat Animasi Objek Sederhana

Deskripsi Project

Membuat animasi 2D sederhana menggunakan Python (Matplotlib) dengan menerapkan minimal dua jenis transformasi (translasi, rotasi, scaling, atau refleksi).

Tujuan Project

- Menerapkan konsep transformasi geometri dalam animasi.
- Menggabungkan beberapa transformasi untuk membuat animasi yang menarik.
- Mengasah kreativitas dalam merancang animasi 2D.

Ketentuan Umum

- ✓ Dikerjakan secara individu/kelompok (2–3 orang).
- ✓ Menggunakan Python dan Matplotlib.
- ✓ Durasi animasi minimal 5–10 detik (≥ 150 frame pada 30 FPS).
- ✓ Menampilkan minimal 2 transformasi.

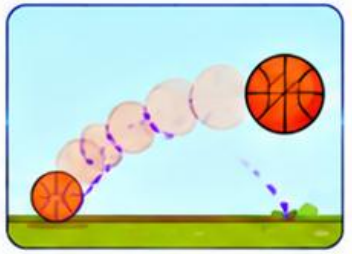
Contoh Ide Animasi



Mobil Bergerak (Translasi)



Kincir Berputar (Rotasi)



Bola Memantul (Translasi + Scaling)

Langkah Pengerjaan Project

1. Tentukan ide animasi.
2. Rancang objek dan parameternya.
3. Implementasikan transformasi (≥ 2 jenis).
4. Atur durasi, FPS, dan timeline.
5. Jalankan, uji, dan perbaiki hasil animasi.

Contoh Kombinasi Transformasi

- **Karakter berjalan:** translasi + rotasi (gerak kaki/tangan).
- **Balon membesar–mengecil:** scaling + translasi.
- **Pesawat terbang:** rotasi (kemiringan) + translasi.
- **Objek memantul:** translasi vertikal + scaling.



Kriteria Penilaian

Aspek Penilaian	Bobot
Penerapan transformasi (min. 2 jenis)	40%
Kreativitas dan desain animasi	25%
Kerapian dan kebenaran kode program	20%
Kualitas hasil animasi (kehalusan, durasi, dan kesseuaian)	15%

Luaran (Deliverables)

- File kode program Python (.py)
- Hasil animasi (video .mp4 atau .gif)

Contoh Output Mini Project



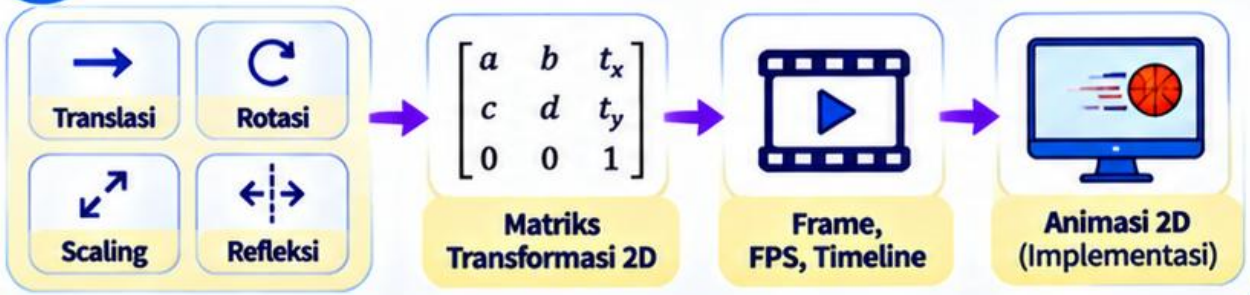
Kesimpulan dan Pengantar Pertemuan 7

Review Materi dan Arah Pembelajaran Berikutnya

Kesimpulan Pertemuan 6

- Transformasi geometri 2D digunakan untuk memanipulasi posisi, orientasi, ukuran, dan bentuk objek.
- Jenis transformasi: translasi (memindahkan), rotasi (memutar), scaling (mengubah ukuran), dan refleksi (mencerminkan).
- Transformasi dapat direpresentasikan dengan matriks 3 × 3 (koordinat homogen) untuk perhitungan yang efisien.
- Animasi 2D dibangun dari urutan frame, dengan FPS (Frames Per Second) yang menentukan kehalusan gerakan, dan timeline untuk mengatur waktunya.
- Telah dilakukan praktikum animasi translasi, rotasi, serta transformasi gabungan, dan pengembangan dalam mini project.

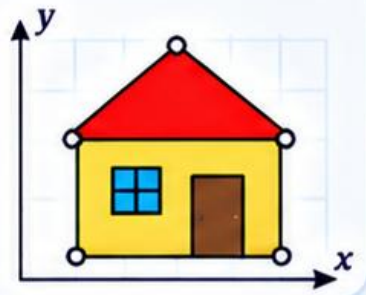
Peta Konsep (Ringkasan)



Pengantar Pertemuan 7

Pemodelan Objek 2D

Pada pertemuan berikutnya, kita akan mempelajari cara membentuk dan memodelkan objek 2D dari primitif grafis, serta teknik manipulasi objek secara lebih terstruktur.



Pokok Bahasan (Pertemuan 7)

- Representasi objek 2D: titik, garis, poligon, dan kurva sederhana.
- Pembentukan objek kompleks dari objek dasar.
- Struktur data untuk menyimpan objek 2D.
- Operasi dasar pada objek (pembuatan, transformasi, dan penggambaran).
- Implementasi dalam Python (Matplotlib).

Persiapan

- Review kembali materi transformasi (translasi, rotasi, scaling, refleksi).
- Siapkan environment Python (Matplotlib) dan contoh kode sebelumnya.
- Bawa ide objek 2D sederhana yang akan dimodelkan pada pertemuan 7 (misal: rumah, mobil, karakter, atau bangun lainnya).